# MLFP mlfp03 — Speaker Notes


- Module 3: Supervised Machine Learning for Building and Deploying Models — Speaker Notes
  - Slide 1: Module 3 Title — Supervised Machine Learning for Building and Deploying Models
  - Slide 2: What You Will Learn
  - Slide 3: Your Journey — 8 Lessons
  - Slide 4: Kailash Engines — Cumulative Map
  - Slide 5: Where We Are
  - Slide 6: Lesson 3.1 — Feature Engineering, ML Pipeline, and Feature Selection
  - Slide 7: The ML Pipeline
  - Slide 8: Statistics vs Machine Learning
  - Slide 9: Data > Models > Hyperparameter Tuning
  - Slide 10: Types of Engineered Features
  - Slide 11: Feature Leakage — The Silent Killer
  - Slide 12: Feature Selection — Three Families
  - Slide 13: Kailash Bridge — FeatureEngineer and FeatureStore
  - Slide 14: Lesson 3.1 Summary
  - Slide 15: Lesson 3.2 — Bias-Variance, Regularisation, and Cross-Validation
  - Slide 16: Bias and Variance — The Darts Analogy
  - Slide 17: Bias-Variance Decomposition
  - Slide 18: The Three Terms
  - Slide 19: Regularisation — Controlling Complexity
  - Slide 20: Bayesian Interpretation of L2
  - Slide 21: Cross-Validation
  - Slide 22: Kailash Bridge — TrainingPipeline
  - Slide 23: Lesson 3.2 Summary
  - Slide 24: Lesson 3.3 — The Complete Supervised Model Zoo
  - Slide 25: No Free Lunch Theorem
  - Slide 26: Support Vector Machines (SVM)
  - Slide 27: K-Nearest Neighbors (KNN)
  - Slide 28: Naive Bayes
  - Slide 29: Decision Trees
  - Slide 30: Random Forests
  - Slide 31: Model Comparison Framework
  - Slide 32: Kailash Bridge — AutoMLEngine
  - Slide 33: Lesson 3.3 Summary
  - Slide 34: Lesson 3.4 — Gradient Boosting Deep Dive
  - Slide 35: Bagging vs Boosting
  - Slide 36: AdaBoost — Conceptual Warmup
  - Slide 37: XGBoost — The Math
  - Slide 38: XGBoost Split Gain Formula
  - Slide 39: LightGBM and CatBoost
  - Slide 40: Boosting Family Comparison
  - Slide 41: Kailash Bridge — Gradient Boosting
  - Slide 42: Lesson 3.4 Summary
  - Slide 43: Lesson 3.5 — Model Evaluation, Imbalance, and Calibration
  - Slide 44: Remember the AML Model?
  - Slide 45: Classification Metrics
  - Slide 46: Regression Metrics
  - Slide 47: Choosing the Right Metric
  - Slide 48: Handling Class Imbalance
  - Slide 49: Focal Loss

- Slide 50: Probability Calibration
- Slide 51: Lesson 3.5 Summary
- Slide 52: Lesson 3.6 — Interpretability and Fairness
- Slide 53: Why Interpretability Matters
- Slide 54: SHAP — Shapley Additive Explanations
- Slide 55: SHAP Variants and Plots
- Slide 56: LIME — Local Interpretable Explanations
- Slide 57: Fairness Metrics
- Slide 58: The Fairness Impossibility Theorem
- Slide 59: ALE — Accumulated Local Effects
- Slide 60: Lesson 3.6 Summary
- Slide 61: Lesson 3.7 — Workflow Orchestration, Model Registry, and Hyperparameter Search
- Slide 62: WorkflowBuilder — Node-Based Pipelines
- Slide 63: Custom Nodes
- Slide 64: Bayesian Hyperparameter Search
- Slide 65: Model Registry — Versioning and Lifecycle
- Slide 66: Model Lifecycle
- Slide 67: Nested Cross-Validation — Unbiased Model Selection
- Slide 68: Lesson 3.7 Summary
- Slide 69: Lesson 3.8 — Production Pipeline, DataFlow, Drift, and Deployment
- Slide 70: DataFlow — Database Persistence
- Slide 71: Async/Await — Quick Primer
- Slide 72: DataFlow — ConnectionManager
- Slide 73: DriftMonitor — Detecting Model Decay
- Slide 74: PSI and KS Test
- Slide 75: Kailash Bridge — DriftMonitor
- Slide 76: Model Cards — Documenting Your Model
- Slide 77: Conformal Prediction — Uncertainty Quantification
- Slide 78: The Full Production Pipeline
- Slide 79: MLOps — CI/CD for Machine Learning
- Slide 80: Lesson 3.8 Summary
- Slide 81: Module 3 — Key Formulas Reference
- Slide 82: Module 3 — Complete Engine Map
- Slide 83: Module 3 — What You Can Now Do
- Slide 84: Module 3 — Assessment
- Slide 85: Exercise Summary
- Slide 86: Common Mistakes to Avoid
- Slide 87: Model Selection Decision Tree
- Slide 88: The Complete Supervised Pipeline
- Slide 89: Coming Up — Module 4
- Slide 90: Module 3 Complete


# Module 3: Supervised Machine Learning for Building and Deploying Models — Speaker Notes

Total time: ~180 minutes (3 hours) Audience: working professionals; instructors must scaffold for both novices and experienced ML practitioners. Use the three-layer markers on the deck (FOUNDATIONS / THEORY / ADVANCED) to pace the room.

---

# Slide 1: Module 3 Title — Supervised Machine Learning for Building and Deploying Models

**Time**: ~2 min **Talking points**:

- Welcome the class back to MLFP. Read the provocation aloud: “Data is more important than models. But the wrong model can still ruin perfect data.”
- This module takes everything from M1 (data pipelines) and M2 (statistics and regression) and builds the complete supervised ML pipeline.
- By the end of 8 lessons, students will train every major supervised model, evaluate honestly, interpret predictions, and deploy to production with drift monitoring.
- Ask the room: “How many of you have trained an ML model before?” Gauge experience and adjust depth accordingly.
- If beginners look confused: “M2 gave you one model — linear regression. Today we give you the whole toolbox and show you how professionals pick the right tool and ship it to production.”
- If experts look bored: “We derive the XGBoost split gain from a second-order Taylor expansion, prove the fairness impossibility theorem, and build a nested cross-validation loop. There is depth here for practitioners too.” **Transition**: “Let us start with what you will be able to do by the end of today.”

---

# Slide 2: What You Will Learn

**Time**: ~2 min **Talking points**:

- Walk through the three layers: FOUNDATIONS (green) — skills everyone leaves with; THEORY (blue) — the math of why it works; ADVANCED (purple) — production engineering.
- Reassure foundations-level students they will not be left behind. Tell advanced students the blue and purple slides contain full mathematical derivations.
- Call out that the lesson markers on every slide tell them whether to engage or rest.
- If beginners look confused: “You do not need to follow every formula to finish the exercises. The foundations track alone lets you train, evaluate, and deploy a real model.”
- If experts look bored: “The blue and purple content goes past most masters-level syllabi — the bias-variance derivation, the XGBoost Newton step, and the Chouldechova impossibility proof.” **Transition**: “Here is the roadmap for our 8 lessons.”

---

# Slide 3: Your Journey — 8 Lessons

**Time**: ~2 min **Talking points**:

- Walk the table at a glance — do not read every cell.
- Progression: features first (3.1), theory of learning (3.2), the model zoo (3.3 and 3.4), honest evaluation (3.5), interpretation (3.6), then engineering it all (3.7 and 3.8).
- Each lesson builds on the last. Skipping a lesson is fine, but the exercises assume prior lessons.
- “By Lesson 3.8, you will have a credit-scoring model in a database, with drift monitoring, a model card, and a promotion gate — the same stack a regulated bank ships.” **Transition**: “To do all that, you will meet a lot of Kailash engines today. Here is the cumulative map.”

---

# Slide 4: Kailash Engines — Cumulative Map

**Time**: ~2 min **Talking points**:

- Show how the engine set grows across MLFP. M3 is the single largest engine introduction in the whole course.
- Call each engine by name: FeatureEngineer, FeatureStore, TrainingPipeline, AutoMLEngine, HyperparameterSearch, ModelRegistry, WorkflowBuilder, DataFlow, DriftMonitor, ModelVisualizer, EnsembleEngine.
- Reassure: we introduce them progressively — never drop all 13 on the room at once.
- If beginners look confused: "Think of these as power tools. We teach the concept by hand first, then show the Kailash engine that automates it."
- If experts look bored: "The engines wrap the usual sklearn, XGBoost, SHAP, and MLflow stack with polars-native APIs and point-in-time correctness. The abstraction is thin." **Transition**: "Before we dive in, let me place M3 on the MLFP map."

---

# Slide 5: Where We Are

**Time**: ~1 min **Talking points**:

- Quick recap. Do not linger. Students who missed prior modules get oriented.
- The key connection: M2's linear regression is the simplest supervised model. M3 adds the full zoo, proper evaluation, interpretability, and production deployment.
- M4 will then pivot to unsupervised learning; M5 brings LLMs and RAG; M6 is alignment and governance. **Transition**: "Lesson 3.1. Features and the pipeline that carries them."

---

# Slide 6: Lesson 3.1 — Feature Engineering, ML Pipeline, and Feature Selection

**Time**: ~1 min **Talking points**:

- Read the subtitle: "Data is more important than models — and features are the language data speaks."
- Feature engineering is where domain expertise meets machine learning. The best model in the world cannot learn from bad features.
- Today in 3.1 we cover the full pipeline, types of features, leakage, and three families of feature selection. **Transition**: "The ML pipeline is bigger than most beginners think. Let us map it."

---

# Slide 7: The ML Pipeline

**Time**: ~2 min **Talking points**:

- Walk through the seven stages: ingestion, preprocessing, feature engineering, model selection, training and evaluation, hyperparameter tuning, deployment.
- Emphasise iteration. In practice, most professional time is spent on data and features, not on picking XGBoost vs LightGBM.
- The callout: the pipeline is a cycle, not a line. Bad evaluation sends you back to features. Production drift sends you back to retraining.
- If beginners look confused: "Imagine a factory. Raw materials come in, get cleaned, get shaped, then assembled, then quality-checked. ML is the same — and sometimes the quality check sends a product back to the start."

- If experts look bored: “The arrows matter. In MLOps language, every edge in this diagram is a place where contracts can break. We formalise those contracts in 3.7 with ModelSignature.” **Transition**: “Before we engineer features, let me draw a line between what we are doing today and what we did in M2.”

---

# Slide 8: Statistics vs Machine Learning

**Time**: ~2 min **Talking points**:

- This distinction is critical. Many professionals confuse the two.
- Statistics asks “why?” — explain the past, coefficients, p-values, confidence intervals, small-to-medium data.
- ML asks “what next?” — predict the future, test error, cross-validation, medium-to-large data.
- Both use the same math but with different goals. M2 taught regression as a statistical model (explaining coefficients). M3 uses it as a prediction machine (minimising test error).
- Singapore example: MAS stress-testing reports use statistical regression — they need the coefficients. Grab’s ETA model uses ML regression — it needs the next prediction to be accurate.
- If beginners look confused: “A doctor explaining why a patient got sick uses statistics. A doctor predicting who will get sick next uses ML.”
- If experts look bored: “Breiman’s 2001 ‘Two Cultures’ paper is the classic reference. The cultures are closer now than then, but the goals still diverge.” **Transition**: “With ML framed as prediction, what matters most for prediction accuracy?”

---

# Slide 9: Data > Models > Hyperparameter Tuning

**Time**: ~2 min **Talking points**:

- The hierarchy: better data and features give the largest impact (10x possible), better model choice gives 2-5x, better hyperparameters give 1.1-1.5x.
- Common mistake: spending days tuning XGBoost when a single well-engineered feature would have helped more than all tuning combined.
- The HDB example: geocoding an address to latitude and longitude (domain knowledge) is worth more than switching from Random Forest to XGBoost, because it unlocks distance-to-MRT and school-count features.
- If beginners look confused: “Give a student a good textbook and an average teacher, they will learn more than a mediocre textbook and a great teacher.”
- If experts look bored: “Andrew Ng’s data-centric AI push formalises this — we will revisit it when we talk about label noise in M4.” **Transition**: “So what does ‘better features’ actually mean? Let us taxonomise.”

---

# Slide 10: Types of Engineered Features

**Time**: ~2 min **Talking points**:

- Walk each type with the HDB example: Temporal (lag price, rolling 3-month mean), Interaction (floor_area x storey_range), Polynomial (floor_area squared), Domain (distance_to_mrt, school_count_1km), Aggregation (avg_price_in_town).
- Temporal features are critical for time-dependent data. Interaction terms capture non-additive effects. Domain features require human expertise — that is why feature engineering is an art as much as a science.
- “For HDB, ‘distance to nearest MRT’ is a domain feature you have to build. No model finds it automatically.”

- If beginners look confused: “A feature is just a column. We are making new columns by combining or transforming the old ones.”
- If experts look bored: “Polynomial features are a linear model’s way of chasing gradient boosting. We will see why trees do this more elegantly in 3.3.” **Transition**: “Features can also be your worst enemy. Here is the silent killer.”

---

## Slide 11: Feature Leakage — The Silent Killer

**Time**: ~2 min **Talking points**:

- Leakage is the most common ML bug in production systems. A model with leakage looks like a breakthrough in testing and fails spectacularly live.
- Classic Singapore example: a loan default model that uses “collections department contacted” as a feature — that field only exists after default, not at application time.
- The prevention rule is simple: time-travel test every feature. At the moment of prediction, would this value be available?
- If beginners look confused: “Imagine predicting tomorrow’s weather using tomorrow’s actual temperature. Of course it works. But it is cheating.”
- If experts look bored: “Point-in-time correctness is what FeatureStore gives you for free. You will see that in two slides.” **Transition**: “Once you have features, how do you decide which to keep?”

---

## Slide 12: Feature Selection — Three Families

**Time**: ~2 min **Talking points**:

- Three families, three tradeoffs.
- Filter methods: fast, model-free, miss interactions. Examples: mutual information, chi-squared, correlation thresholds.
- Wrapper methods: find the best subset, cost N model trainings. Examples: forward selection, backward elimination, RFE.
- Embedded methods: built into the model itself. Examples: L1 sparsity in Lasso, tree-based importance. These are the practical default.
- In the exercise, students apply all three and compare which features each selects. They will find little overlap, which is the key lesson.
- If beginners look confused: “Filter is like sorting groceries by price. Wrapper is like trying every recipe to see which tastes best. Embedded is a chef who knows the recipe picks the ingredients.”
- If experts look bored: “RFE with cross-validation is still a reasonable practical default for mid-sized tabular problems. L1 tends to underperform when features are highly collinear.” **Transition**: “Kailash packages all three families for you. Here is the bridge.”

---

## Slide 13: Kailash Bridge — FeatureEngineer and FeatureStore

**Time**: ~2 min **Talking points**:

- FeatureEngineer wraps all three selection families in one API. Students do not need to hand-wire mutual information and RFE.
- FeatureStore ensures point-in-time correctness: when you request features for a prediction made at 2025-03-15, you get values as of that date, not current values. Temporal leakage becomes impossible.
- These two engines handle the entire feature stage of the pipeline.

- "For the exercise, students use FeatureEngineer to generate and select features for HDB prices, then save them to a FeatureStore for reuse across models." **Transition**: "That is Lesson 3.1. Here is the summary."

---

## Slide 14: Lesson 3.1 Summary

**Time**: ~1 min **Talking points**:

- Quick recap: pipeline, feature types, leakage, three selection families, FeatureEngineer and FeatureStore.
- The exercise gives hands-on practice with all three selection families on real HDB data.
- Key takeaway for beginners: "Data beats models. Put your time into features first." **Transition**: "Now to the single most important theoretical idea in ML. Lesson 3.2: bias and variance."

---

## Slide 15: Lesson 3.2 — Bias-Variance, Regularisation, and Cross-Validation

**Time**: ~1 min **Talking points**:

- The fundamental tradeoff in all of ML. This lesson gives the theoretical foundation for every model choice in the rest of the module.
- If students understand bias-variance, they understand why we need regularisation, cross-validation, and model comparison.
- Read the subtitle line to set the stakes. **Transition**: "Let us start with the most intuitive explanation ever drawn."

---

## Slide 16: Bias and Variance — The Darts Analogy

**Time**: ~2 min **Talking points**:

- The darts analogy is the clearest mental picture. Four quadrants: high bias / high variance, high bias / low variance, low bias / high variance, low bias / low variance.
- High bias = you are aiming wrong. High variance = you are shaky. The ideal is low both.
- Every model in the zoo today has a different bias-variance profile — that is why model selection matters.
- If beginners look confused: "A calm but wrong shooter has high bias. A shaky but well-aimed shooter has high variance. ML asks you to train both your aim and your steadiness."
- If experts look bored: "The analogy extends to ensembles: bagging reduces variance without touching bias; boosting reduces bias while risking variance. 3.3 and 3.4 are the two sides of this coin." **Transition**: "Here is the same idea in math."

---

## Slide 17: Bias-Variance Decomposition

**Time**: ~3 min **Talking points**:

- Full derivation: E[(y - y_hat)^2] = Bias^2 + Variance + sigma^2.
- Walk through the cancellation: cross terms vanish because noise is zero-mean and independent of the model, and the variance term is zero-mean by definition.
- The three pieces are the only three sources of prediction error — period.

- For beginners: “We split total error into three parts — how wrong we are on average (bias), how much predictions wobble (variance), and noise we cannot fix.”
- For experts: note this decomposition is for squared loss; the decomposition differs for 0/1 loss and cross-entropy, and there is a whole literature on those. **Transition**: “Let me summarise what each term means.”

---

# Slide 18: The Three Terms

**Time**: ~2 min **Talking points**:

- Bias^2 — systematic error. Caused by model being too simple (linear model on non-linear data).
- Variance — sensitivity to training data. Caused by model being too flexible (deep tree overfitting).
- sigma^2 — irreducible noise. You cannot fix it with any model. This is the ceiling.
- This table is the conceptual anchor for the rest of the module. Every model comparison comes back to this: where does it sit on the bias-variance spectrum?
- “Regularisation is next because it is the primary tool for controlling this tradeoff.” **Transition**: “So how do we control complexity? With a budget.”

---

# Slide 19: Regularisation — Controlling Complexity

**Time**: ~3 min **Talking points**:

- L1 (Lasso): drives coefficients to exactly zero. Diamond-shaped constraint region. Corners on the axes — that is why solutions are sparse.
- L2 (Ridge): shrinks coefficients toward zero but not all the way. Circle-shaped constraint region. No corners, no exact zeros.
- Elastic Net: alpha * L1 + (1-alpha) * L2. Practical default when features are correlated — gives you both sparsity and stability.
- If beginners look confused: “Think of regularisation as a budget. The model has a spending limit on coefficient size, so it cannot make any single feature too dominant.”
- If experts look bored: “The L1/L2 geometry is the cleanest proof that lasso gives sparsity. For correlated features, lasso can be unstable — Elastic Net fixes that with the quadratic term.” **Transition**: “L2 has a surprising Bayesian interpretation that connects back to M2.”

---

# Slide 20: Bayesian Interpretation of L2

**Time**: ~2 min **Talking points**:

- Advanced slide. Call it out as THEORY (blue marker).
- The key insight: L2 regularisation is exactly equivalent to placing a Gaussian prior on the coefficients and taking the posterior maximum.
- Regularisation is not arbitrary — it encodes a prior belief that most features have small effects.
- For experts: mention that the full posterior also gives uncertainty estimates. This connects to conformal prediction in Lesson 3.8.
- If beginners look confused: “The math here is optional. The takeaway is: regularisation is mathematically principled, not a hack.” **Transition**: “Whether you use L1, L2, or Elastic Net, how do you know the model actually generalises? You cross-validate.”

---

# Slide 21: Cross-Validation

**Time**: ~2 min **Talking points**:

- Cross-validation is how we estimate real-world performance before we ship.
- k-fold: split into k folds, train on k-1, validate on 1, rotate. Default k = 5 or 10.
- Stratified k-fold: preserves class proportions — essential for imbalanced data.
- Time-series split: walk-forward validation — no future data leakage. Critical for financial and HDB data.
- GroupKFold: when observations are grouped (same patient, same company) — prevents leakage from grouped samples.
- Nested CV: outer loop for model selection, inner loop for hyperparameters. Gold standard but expensive; covered in detail in 3.7.
- "For financial professionals in the room: if you ever use k-fold on time-series data, you are leaking the future into the past. Always use walk-forward." **Transition**: "Kailash's TrainingPipeline gets this right for you by default."

---

# Slide 22: Kailash Bridge — TrainingPipeline

**Time**: ~2 min **Talking points**:

- TrainingPipeline is the workhorse engine for M3. It handles the boilerplate of cross-validation correctly.
- The key selling point: it fits preprocessing inside the CV loop. Most beginners get this wrong when doing it manually — they scale on the full dataset first, then split, which leaks statistics from the validation fold into training.
- Shows the polars-native API. Students should be comfortable with polars from M1.
- "Every model in this module is trained through TrainingPipeline. You will not hand-wire k-fold after today." **Transition**: "Lesson 3.2 summary, then we jump into the zoo."

---

# Slide 23: Lesson 3.2 Summary

**Time**: ~1 min **Talking points**:

- Quick recap: bias and variance, L1/L2/Elastic Net, CV strategies, TrainingPipeline.
- The exercise makes the theory tangible: students see train/test error curves diverge as complexity increases, then see regularisation pull them back together.
- Ask: "Any questions before we enter the model zoo?" **Transition**: "Lesson 3.3. Meet every classical supervised model."

---

# Slide 24: Lesson 3.3 — The Complete Supervised Model Zoo

**Time**: ~1 min **Talking points**:

- The No Free Lunch theorem says no model wins on all problems. This lesson teaches every major supervised algorithm so students can match the algorithm to the problem.
- We cover SVM, KNN, Naive Bayes, Decision Trees, and Random Forests. XGBoost and the boosting family get their own lesson (3.4). **Transition**: "Let us start with why the zoo exists at all."

---

# Slide 25: No Free Lunch Theorem

**Time**: ~2 min **Talking points**:

- No Free Lunch (Wolpert 1996): averaged over all possible problems, every algorithm performs identically. No universal winner.
- This is why we must know the full zoo — to pick the right model for each problem class.
- For experts: in practice, real-world data has structure that some algorithms exploit better than others. So some models tend to win on specific structures (gradient boosting on tabular, CNNs on images, transformers on text).
- If beginners look confused: "Different tools for different jobs. You would not use a hammer for a screw, even if both can hit things." **Transition**: "Let us meet the zoo. First: SVM."

---

# Slide 26: Support Vector Machines (SVM)

**Time**: ~3 min **Talking points**:

- SVM finds the hyperplane with the widest margin between classes. Maximise 2 / ||w|| subject to the constraint that every point is on the correct side with margin at least 1.
- Kernel trick: replace dot products with kernel functions — lets SVM handle non-linear boundaries by implicitly working in high-dimensional space. RBF and polynomial kernels are the main non-linear choices.
- Soft margin: C parameter controls how much you penalise margin violations. Low C = wide margin, more tolerance. High C = narrow margin, zero tolerance.
- When to use: high-dimensional data with clear class separation. Text classification with TF-IDF features is still a solid SVM use case.
- SVM scales poorly to large datasets — O(n^2) to O(n^3) training — so it is rarely used on billion-row datasets.
- If beginners look confused: "Imagine drawing a line between two groups of points. SVM finds the line with the widest buffer zone."
- If experts look bored: "The dual formulation and KKT conditions give you support vectors as the only points that matter — a strong sparsity property." **Transition**: "Next: the simplest algorithm imaginable."

---

# Slide 27: K-Nearest Neighbors (KNN)

**Time**: ~2 min **Talking points**:

- KNN has no training phase. It just memorises the data. To predict, find the k closest training points and vote (classification) or average (regression).
- Distance metrics matter: Euclidean for continuous, Manhattan for grid-like, cosine for text and high-dimensional.
- Curse of dimensionality: in high dimensions, all points become roughly equidistant. KNN breaks down.
- k selection: small k = noisy, overfit. Large k = smooth, underfit. Tune via CV.
- When to use: small data, interpretable decision boundaries, no need for a training step.
- If beginners look confused: "Tell me who your five nearest neighbours are, and I will tell you who you are."
- If experts look bored: "KNN is asymptotically optimal as n goes to infinity with k growing properly, but the convergence rate is terrible in high dimensions. Local methods lose to global methods there." **Transition**: "Now the classical Bayesian classifier."

---

# Slide 28: Naive Bayes

**Time**: ~2 min **Talking points**:

- Three variants: GaussianNB (continuous features), MultinomialNB (counts, e.g. word frequencies), BernoulliNB (binary features).
- The "naive" assumption: features are conditionally independent given the class. Almost always wrong — features are correlated.
- It still works well because the classifier only needs to get the ranking right, not the exact probabilities.
- Classic application: text classification. Spam filtering. Language ID. Each word treated as independent, which is wrong but effective.
- Connects to M2's Bayesian thinking: prior * likelihood / evidence.
- Good baseline before trying complex models. Fast to train, fast to predict.
- If beginners look confused: "Assume every clue is independent, multiply them together, and pick the most likely explanation. It is wrong in theory but often right in practice."
- If experts look bored: "The zero-frequency problem and Laplace smoothing are still relevant. And for calibrated probabilities, you need Platt or isotonic on top — we cover that in 3.5." **Transition**: "Now the most interpretable model: trees."

---

# Slide 29: Decision Trees

**Time**: ~3 min **Talking points**:

- Trees recursively split the feature space. At each node, pick the feature and threshold that best separates the classes.
- Splitting criteria: Gini impurity (G = 1 - sum(p_i^2)) or entropy/information gain. Both give nearly identical splits in practice; Gini is slightly faster to compute.
- Pruning is essential. Pre-pruning: max_depth, min_samples_split, min_samples_leaf. Post-pruning: grow the tree fully, then collapse back.
- Trees are the most interpretable model: you can draw the tree and explain every decision. For regulated industries, this is valuable.
- Weakness: a single tree overfits badly. That is why we need forests (next slide) and boosting (3.4).
- Singapore example: MAS-regulated credit models often use decision trees exactly because the decision path is auditable.
- If beginners look confused: "A tree is a flowchart. 'Is income above 80k? Yes -> is debt ratio below 30%?' and so on."
- If experts look bored: "CART, ID3, C4.5 are historical. In practice, sklearn's implementation covers Gini and entropy. The real innovation today is how trees are combined — bagging and boosting." **Transition**: "And here is how we tame the variance of trees: vote them."

---

# Slide 30: Random Forests

**Time**: ~2 min **Talking points**:

- Random Forest = many decision trees trained on different bootstrap samples with random feature subsets. Predictions are the majority vote (classification) or mean (regression).
- Two sources of randomness: bootstrap sampling (bagging) and feature subsampling at each split. Together they decorrelate the trees.
- Out-of-bag (OOB) error: about 36.8% of samples are not in each bootstrap sample (because (1 - 1/n)^n -> 1/e). Those samples serve as a free validation set.

- Feature importance: measured by how much each feature reduces impurity across all trees.
- Random Forest is the "Swiss army knife" of ML. Works well on almost everything with minimal tuning.
- The connection to bias-variance: this is how ensembles overcome the single-tree overfitting problem — bagging reduces variance without touching bias.
- If beginners look confused: "Ask 500 slightly biased experts and take the vote. Their individual errors cancel out."
- If experts look bored: "OOB is a beautiful free lunch — you get cross-validation without the computational cost. For production, it often replaces held-out CV." **Transition**: "So we have 5 model families. How do we compare them?"

---

# Slide 31: Model Comparison Framework

**Time**: ~2 min **Talking points**:

- This table is the cheat sheet students will reference for months.
- Four axes: accuracy, interpretability, training speed, data size scaling.
- The right model depends on the use case, not just accuracy. A bank regulator wants interpretability (tree). A Kaggle competitor wants accuracy (boosting). A spam filter wants speed (Naive Bayes). A small-data medical study wants KNN.
- "There is no universally best model. There is only the best model for your constraints."
- Ask the room: "If you were building the AML model from the opening story, which model family would you start with, and why?" **Transition**: "To compare 5 models fairly, you need identical splits and preprocessing. Kailash automates that."

---

# Slide 32: Kailash Bridge — AutoMLEngine

**Time**: ~2 min **Talking points**:

- AutoMLEngine automates the "try every model" step.
- Key benefit: consistency. Same splits, same preprocessing across all models. This prevents the common mistake of comparing models on different data splits.
- Under the hood: tries SVM, KNN, Naive Bayes, trees, RF, and optionally boosting. Returns a ranked leaderboard.
- Students use AutoMLEngine in the exercise to compare all 5 model families on e-commerce customer classification.
- "Do not use AutoMLEngine as a black box. Use it to build the leaderboard, then dig into the top 2 or 3 manually." **Transition**: "Lesson 3.3 summary."

---

# Slide 33: Lesson 3.3 Summary

**Time**: ~1 min **Talking points**:

- Quick recap: the zoo (SVM, KNN, NB, Trees, RF), No Free Lunch, comparison framework, AutoMLEngine.
- The exercise forces students to use consistent evaluation (same CV splits) and justify their model selection with data evidence, not opinion.
- Ask: "Any questions about the classical zoo before we go deep on gradient boosting?" **Transition**: "Lesson 3.4. The dominant algorithm for tabular data."

---

# Slide 34: Lesson 3.4 — Gradient Boosting Deep Dive

**Time**: ~1 min **Talking points**:

- Gradient boosting is the most important family for tabular data. XGBoost, LightGBM, and CatBoost dominate Kaggle and production systems at banks, e-commerce, and insurance.
- This lesson goes deep into the math of how and why they work.
- For professionals: if your tabular prediction problem is not solved by gradient boosting, you should seriously question whether the problem is solvable at all. **Transition**: "Start with the conceptual distinction: bagging versus boosting."

---

# Slide 35: Bagging vs Boosting

**Time**: ~2 min **Talking points**:

- Bagging (Random Forest): train many independent models in parallel on different bootstrap samples, then average. Reduces variance.
- Boosting (XGBoost): train models sequentially, each one correcting the errors of the previous. Reduces bias.
- Bagging and boosting are complementary ensemble strategies, not competitors.
- If beginners look confused: "Bagging asks many experts and takes a vote. Boosting asks one expert, then asks a specialist to fix the mistakes, then another specialist to fix the remaining mistakes."
- If experts look bored: "Bagging's variance-reduction effect comes from averaging decorrelated estimators. Boosting's bias reduction comes from descending the empirical loss. Different objectives, different guarantees." **Transition**: "Historical warmup: AdaBoost."

---

# Slide 36: AdaBoost — Conceptual Warmup

**Time**: ~2 min **Talking points**:

- AdaBoost (Freund and Schapire, 1996) is the original boosting algorithm. Reweight misclassified samples so the next model focuses on the hard ones.
- Students understand the "focus on mistakes" idea before we add gradient descent.
- In practice, nobody uses AdaBoost anymore. But the intuition transfers directly to XGBoost.
- "AdaBoost is the grandparent of modern boosting. Respect it, then move on." **Transition**: "XGBoost formalises the idea with gradients and Newton steps."

---

# Slide 37: XGBoost — The Math

**Time**: ~3 min **Talking points**:

- XGBoost (Chen and Guestrin, 2016) introduced a critical innovation: use the second-order Taylor expansion of the loss function, not just the first-order gradient.
- Most gradient boosting implementations only use the gradient (1st order). XGBoost adds the Hessian — that is like adding Newton's method to gradient descent.
- Walk through: objective = sum of losses + regularisation. Second-order expansion gives a quadratic in the leaf weights, solvable in closed form.
- For beginners: "XGBoost uses both the slope and the curvature to find the best correction. Steeper descent, fewer iterations."

- For experts: note this is why XGBoost supports arbitrary differentiable loss functions — you just supply the gradient and Hessian. **Transition**: "The clever consequence is the split gain formula."

---

# Slide 38: XGBoost Split Gain Formula

**Time**: ~3 min **Talking points**:

- Gain = 1/2 * [G_L^2 / (H_L + lambda) + G_R^2 / (H_R + lambda) - (G_L + G_R)^2 / (H_L + H_R + lambda)] - gamma.
- This is the formula students need to understand, not memorise. Walk through term by term: left child score + right child score - parent score - complexity penalty.
- The key insight: the gain formula naturally incorporates regularisation.
  - Lambda penalises extreme leaf weights (L2 on the leaves).
  - Gamma penalises splits that do not improve the objective enough (minimum split loss).
- Together they prevent overfitting without needing a separate pruning pass.
- If beginners look confused: "The math is advanced. The takeaway: XGBoost has regularisation baked into its split decisions. That is why it rarely overfits."
- If experts look bored: "G and H come from the task-specific loss, so the split decision is task-aware. Classification gets different splits than regression even on the same data." **Transition**: "Two other boosting flavours you should know."

---

# Slide 39: LightGBM and CatBoost

**Time**: ~3 min **Talking points**:

- LightGBM (Microsoft, 2017): histogram-based split finding, leaf-wise growth, GOSS (Gradient-based One-Side Sampling). Typically fastest on large datasets.
- CatBoost (Yandex, 2018): ordered boosting — prevents target leakage in categorical encoding. Native categorical feature support — no manual encoding required.
- GOSS: keep top-a% gradient samples, randomly sample b% of small-gradient samples. Gives a stochastic speedup without losing much accuracy.
- Leaf-wise growth: split the leaf with highest gain, not level by level. More accurate but risks overfitting deep trees.
- In practice: try all three. LightGBM for speed, XGBoost for documentation and community, CatBoost when categorical features dominate.
- Singapore example: for a dataset with 'HDB town', 'flat model', 'storey range' all as categoricals, CatBoost often saves significant preprocessing time.
- If beginners look confused: "They are cousins. Same idea, different optimisations. Pick one and be consistent."
- If experts look bored: "GOSS bias-corrects for the subsampling. Ordered boosting prevents the 'prediction shift' problem in target encoding. Real innovations, not just speedups." **Transition**: "Side by side comparison."

---

# Slide 40: Boosting Family Comparison

**Time**: ~2 min **Talking points**:

- Quick reference table.
- XGBoost: most widely used, best documentation, community.

- LightGBM: fastest training, good for large datasets.
- CatBoost: best categorical handling, most out-of-the-box.
- In Kaggle competitions, LightGBM and XGBoost alternate as winners. CatBoost shines when categoricals dominate.
- All three are excellent. The differences are marginal on most problems.
- "Your choice is less about accuracy than about infrastructure. Which one does your production team already use?" **Transition**: "Kailash gives you a unified interface for all three."

---

## Slide 41: Kailash Bridge — Gradient Boosting

**Time**: ~2 min **Talking points**:

- TrainingPipeline provides a unified interface for XGBoost, LightGBM, and CatBoost.
- Students do not need to learn three different APIs. Same fit/predict/evaluate across all three.
- The exercise uses this to compare all three on credit scoring data with the same evaluation.
- "You write the code once. TrainingPipeline swaps the backend." **Transition**: "Lesson 3.4 summary."

---

## Slide 42: Lesson 3.4 Summary

**Time**: ~1 min **Talking points**:

- Recap: bagging vs boosting, AdaBoost warmup, XGBoost math, split gain, LightGBM and CatBoost, TrainingPipeline.
- Transition to evaluation. Now that students can train every model, they need to know how to evaluate honestly. That is Lesson 3.5.
- "We now have a lot of models. The next question: how do we know any of them are actually good?" **Transition**: "Back to the AML story from the opening."

---

## Slide 43: Lesson 3.5 — Model Evaluation, Imbalance, and Calibration

**Time**: ~1 min **Talking points**:

- This lesson is where theory meets reality. Students learn why accuracy can lie, how to handle imbalanced data, and how to calibrate model confidence.
- The AML case from M1's opening is the running example: 99.9% accuracy was useless because the wrong metric was used. **Transition**: "Let me remind everyone of the story."

---

## Slide 44: Remember the AML Model?

**Time**: ~2 min **Talking points**:

- Bring back the opening case. A Singapore bank trained an AML (anti-money-laundering) model that reported 99.9% accuracy.
- Walk the class through the math: if 0.001% of transactions are truly suspicious, a "predict all negative" model is 99.999% accurate. And completely useless.
- "The model missed every money laundering case. And the dashboard said it was perfect."

- Now students have the vocabulary (from Lessons 3.2-3.4) to understand why accuracy failed. We need metrics that penalise missing the positives.
- If beginners look confused: "If you predicted 'no rain' every day in Singapore, you would be right 80% of the time. That does not make you a useful weather forecaster."
- If experts look bored: "This is the imbalanced-class failure mode. We formalise it with precision, recall, and PR-AUC next." **Transition**: "The full metrics taxonomy."

---

# Slide 45: Classification Metrics

**Time**: ~3 min **Talking points**:

- Walk each metric with the AML example.
- Precision: of the transactions we flagged, how many were really suspicious? TP / (TP + FP).
- Recall (sensitivity, true positive rate): of all suspicious transactions, how many did we catch? TP / (TP + FN).
- F1: harmonic mean of precision and recall. Balances the two.
- Specificity (true negative rate): of all clean transactions, how many did we correctly let through?
- ROC-AUC: area under the ROC curve. Measures overall ranking quality across all thresholds.
- Log loss: probability-based metric. Penalises confident wrong predictions more than accuracy does.
- Precision-Recall curve: more informative than ROC for imbalanced data.
- For the AML case: recall matters most. Missing a money laundering case has catastrophic regulatory consequences (a MAS fine dwarfs the cost of investigating false positives).
- If beginners look confused: "Precision is how often your alarm is right. Recall is how often the alarm actually goes off when it should."
- If experts look bored: "PR-AUC beats ROC-AUC when positive class is under 5%. And log loss is the only metric here that is a proper scoring rule — which we return to under calibration." **Transition**: "Regression has its own set of metrics."

---

# Slide 46: Regression Metrics

**Time**: ~2 min **Talking points**:

- R-squared from M2 now has company.
- MAE (mean absolute error): easy to interpret, robust to outliers. "On average, we are off by X dollars."
- MSE (mean squared error): punishes large errors. Sensitive to outliers.
- RMSE: square root of MSE. Same units as the target. Most common.
- MAPE (mean absolute percentage error): easiest to explain to non-technical stakeholders ("our predictions are off by 5% on average"). Breaks when actuals are near zero.
- For HDB prediction: report RMSE in dollars, MAPE in percent. RMSE tells you the spread; MAPE tells business what to expect.
- If beginners look confused: "Different metrics penalise different mistakes. Pick the one that matches your cost."
- If experts look bored: "MAPE is asymmetric — a 50% over-prediction costs more than a 50% under-prediction. Use sMAPE or MASE if you care." **Transition**: "So how do you pick a metric?"

---

# Slide 47: Choosing the Right Metric

**Time**: ~2 min **Talking points**:

- There is no default metric. The right metric depends on the business cost of each type of error.
- Ask students: "In the AML case, what is the cost of a false negative? What is the cost of a false positive?"
- False negative: a missed money laundering case -> MAS fine in the millions.
- False positive: an investigator opens a case on a clean transaction -> perhaps $200 of staff time.
- Asymmetry ratio: 10,000:1. That ratio drives the metric choice — you weight recall extremely heavily.
- Generalise: every business problem has a cost matrix. Translate it into the metric.
- If beginners look confused: "Ask yourself: what is the cost of each kind of mistake? Then pick the metric that punishes expensive mistakes more."
- If experts look bored: "You can go further and train with a custom loss encoding the cost matrix directly. XGBoost supports this via weighted samples." **Transition**: "Imbalance is the scenario where metric choice matters most. Let us look at the techniques."

---

# Slide 48: Handling Class Imbalance

**Time**: ~2 min **Talking points**:

- Three approaches: resampling, cost-sensitive learning, and loss function design.
- SMOTE (Synthetic Minority Over-sampling Technique) is the most famous. Generates synthetic minority samples by interpolating between real ones. Has serious limitations: boundary samples, high-dimensional spaces, and dangerous in regulated contexts because it manufactures 'transactions' that never happened.
- Class weights: simply tell the model to pay more for errors on the minority class. Loss gets weighted by class frequency. This is the practical default.
- Undersampling the majority: cheap, throws away data. Use when majority is so large that sampling is necessary.
- For the AML case: cost-sensitive learning with high weight on false negatives is the right approach. SMOTE is a non-starter — you cannot fabricate money laundering transactions and present them to a regulator.
- If beginners look confused: "Imbalance means the model learns to always predict the majority because that is safe. Fix this by telling it: the minority matters more."
- If experts look bored: "SMOTE's boundary-region failure mode is well documented. ADASYN and BorderlineSMOTE try to fix it. Class weights still outperform in most benchmarks." **Transition**: "One more tool: focal loss."

---

# Slide 49: Focal Loss

**Time**: ~2 min **Talking points**:

- Focal loss was invented for object detection (Lin et al. 2017, RetinaNet). Most anchor boxes in an image are background — extreme imbalance.
- Formula: FL(p_t) = -alpha_t * (1 - p_t)^gamma * log(p_t).
- The (1 - p_t)^gamma factor down-weights easy examples. If the model is confident and correct, that example contributes almost nothing to the loss.
- The insight transfers perfectly to tabular imbalance: most examples are easy majority class, and the model wastes capacity on them.
- For beginners: "Focal loss tells the model: stop celebrating getting the easy cases right. Focus on the hard ones."
- For experts: gamma = 2 is the standard. Alpha handles the class weight. Focal loss works with any differentiable loss and can be dropped into XGBoost as a custom objective. **Transition**: "Even with the right metric and imbalance handled, probabilities need to be trustworthy."

---

# Slide 50: Probability Calibration

**Time**: ~2 min **Talking points**:

- Calibration asks: when the model says 70% probability, does the event actually happen 70% of the time?
- Critical for professionals. A bank that uses model probabilities to set interest rates needs those probabilities to be correct, not just well-ranked.
- Two calibration techniques:
  - Platt scaling: fit a logistic regression on top of model outputs. Simple, assumes sigmoid shape.
  - Isotonic regression: non-parametric. More flexible, needs more data.
- The reliability diagram (calibration plot) shows predicted vs actual probability. A well-calibrated model follows the diagonal. Above the line means underconfident; below means overconfident.
- Proper scoring rule: Brier score = mean squared error of probabilities. Lower is better. Use this to measure calibration quality.
- Singapore example: MAS expects calibrated probabilities in credit-scoring models used for loan decisions. Platt scaling is the standard post-hoc fix.
- If beginners look confused: "A weather forecaster who says '80% rain' should be right 80% of the time. If they are only right 50% of the time, they are miscalibrated."
- If experts look bored: "XGBoost is famously uncalibrated out of the box because the tree outputs are log-odds-like but not well-scaled. Always calibrate post-hoc for decision use." **Transition**: "Lesson 3.5 summary."

---

# Slide 51: Lesson 3.5 Summary

**Time**: ~1 min **Talking points**:

- Recap: the metrics taxonomy, imbalance strategies, focal loss, Platt and isotonic calibration.
- The exercise forces students to confront imbalanced data and discover why accuracy lies. They calibrate the model and see the Brier score improve.
- "Two lessons left in the model-science half: interpretability and fairness, then we pivot fully to engineering." **Transition**: "Lesson 3.6: not just accurate, but explainable and fair."

---

# Slide 52: Lesson 3.6 — Interpretability and Fairness

**Time**: ~1 min **Talking points**:

- Interpretability is not optional in regulated industries (finance, healthcare).
- This lesson covers SHAP, LIME, ALE, and fairness metrics.
- For professionals: regulators increasingly require model explainability. The EU AI Act and MAS FEAT are real compliance requirements, not theoretical. **Transition**: "Start with the 'why'."

---

# Slide 53: Why Interpretability Matters

**Time**: ~2 min **Talking points**:

- Three reasons: regulation, debugging, and trust.
- Regulation: MAS FEAT (Fairness, Ethics, Accountability, Transparency) is directly relevant for Singapore-based financial professionals. The EU AI Act classifies credit scoring and hiring as high-risk systems requiring explainability.

- Debugging: SHAP values have caught data leakage bugs that no other technique revealed. If one feature dominates the explanation unexpectedly, that is a red flag.
- Trust: stakeholders will not act on predictions they cannot understand. A loan officer needs to tell the applicant why their loan was denied.
- For a professional audience, the regulatory angle is the strongest motivator.
- If beginners look confused: "If the model cannot explain itself, the model cannot ship in a bank. Period."
- If experts look bored: "Shapley values are not just nice to have. They are provably unique under the Shapley axioms, which matters legally — no competing attribution method has equivalent guarantees." **Transition**: "SHAP is the modern gold standard."

---

# Slide 54: SHAP — Shapley Additive Explanations

**Time**: ~3 min **Talking points**:

- SHAP (Lundberg and Lee, 2017) applies Shapley values from game theory to feature attribution.
- The Shapley value is the only attribution method that satisfies all four axioms simultaneously: efficiency, symmetry, dummy, and linearity. This is a theorem from game theory (Shapley, 1953).
- Formula: phi_i = sum over S of [|S|! * (|F| - |S| - 1)! / |F|!] * [f(S union {i}) - f(S)].
- It looks scary. The intuition is simple: for every possible team of features, measure how much feature i improves the prediction when added. Average over all possible teams, weighted by how many ways each team can be formed.
- For beginners: "Shapley values are like splitting a restaurant bill fairly based on what each person ordered. Every diner's share depends on what everyone else ate."
- For experts: "The weighted sum is the marginal contribution of feature i averaged over all 2^N feature coalitions. It is the unique solution satisfying the four axioms." **Transition**: "Computing Shapley values exactly is exponential. SHAP has clever shortcuts."

---

# Slide 55: SHAP Variants and Plots

**Time**: ~2 min **Talking points**:

- TreeSHAP: efficient exact computation for tree-based models (trees, RF, XGBoost, LightGBM, CatBoost). Polynomial time. The practical choice for our model zoo.
- KernelSHAP: model-agnostic approximation via weighted linear regression. Slower but works on any model.
- DeepSHAP: approximations for deep neural networks. Covered later when we reach deep learning.
- Four standard plots:
  - Summary plot: global feature importance ordered by impact.
  - Dependence plot: how a single feature affects predictions across its range.
  - Waterfall plot: breakdown of a single prediction, one feature at a time.
  - Force plot: interactive version of waterfall, great for dashboards.
- The summary plot is most useful for global understanding. The waterfall plot is most useful for explaining individual predictions to stakeholders (including regulators).
- "For the exercise, students generate summary and waterfall plots for the credit scoring model and present them as they would to a compliance team." **Transition**: "SHAP has a cousin: LIME."

---

# Slide 56: LIME — Local Interpretable Explanations

**Time**: ~2 min **Talking points**:

- LIME (Ribeiro, Singh, Guestrin, 2016): perturb the input, fit a local linear model to the perturbed predictions. "For THIS prediction, these features mattered most."
- Complementary to SHAP. Answers the same question but with a different mechanism.
- Key difference: SHAP gives globally consistent attributions. LIME can give different explanations for similar instances, which is a weakness for compliance use cases.
- For regulatory compliance, SHAP is preferred because of its theoretical guarantees. LIME still has a role for quick debugging and image explanations.
- If beginners look confused: "LIME is like asking: if I wiggle this input a bit, which direction does the model care about most? That gives you a local story for one prediction."
- If experts look bored: "LIME's local-linear assumption can mislead on strongly non-linear models. SHAP does not share this weakness because of its game-theoretic grounding." **Transition**: "Interpretability is half the story. The other half is fairness."

---

# Slide 57: Fairness Metrics

**Time**: ~2 min **Talking points**:

- Three standard fairness metrics.
- Disparate impact: ratio of selection rates between groups. P(Y=1 | G=minority) / P(Y=1 | G=majority). Should be above 0.8 under the US four-fifths rule.
- Equalized odds: TPR and FPR equal across groups. The model makes errors at equal rates.
- Calibration parity: predicted probabilities equally reliable across groups. A 70% prediction means 70% for every group.
- For the credit scoring example: does the model deny loans to one demographic at a higher rate than another? Is a 70% predicted probability equally accurate across ethnicities?
- Singapore context: MAS FEAT explicitly requires demographic fairness testing in AI systems used for credit or insurance decisions.
- If beginners look confused: "Fairness asks: does the model treat every group equally? And there are several ways to measure 'equally'."
- If experts look bored: "Impossibility results tell us these three criteria cannot all hold simultaneously except in trivial cases. We hit that next." **Transition**: "Here is the uncomfortable theorem."

---

# Slide 58: The Fairness Impossibility Theorem

**Time**: ~2 min **Talking points**:

- Chouldechova (2017) and Kleinberg et al. (2016) proved: you cannot simultaneously satisfy demographic parity, equalized odds, AND calibration across groups — unless base rates are identical across groups.
- This is one of the most important results in ML fairness. It means every deployed model makes a fairness tradeoff, whether the team acknowledges it or not.
- The responsible approach: choose the criterion that matches the use case, measure it, document it, and justify the choice to stakeholders.
- Singapore example: for credit models, calibration parity is usually the priority (the probability must mean the same thing everywhere). For hiring, equalized odds often matters more (false negatives and false positives should not fall harder on one group).
- If beginners look confused: "There is no perfect fairness. You must pick which kind of unfairness you can live with — and justify it."
- If experts look bored: "The proof uses the confusion matrix decomposition. If base rates differ and TPR/FPR are equal across groups, calibration cannot hold. Elegant and troubling." **Transition**: "One more interpretability tool for the advanced students."

---

# Slide 59: ALE — Accumulated Local Effects

**Time**: ~2 min **Talking points**:

- Advanced slide (purple marker). For experts and curious beginners.
- ALE is a sophisticated alternative to partial dependence plots (PDP).
- The key insight: when floor area and number of rooms are correlated, PDP evaluates the model at impossible combinations (large area but 1 room). The resulting curve is misleading.
- ALE avoids this by using conditional differences — it only evaluates the model on combinations that actually occur in the data.
- Use when features are correlated. Default to PDP when they are not.
- "For HDB data, floor area and number of rooms are obviously correlated. ALE is the right choice." **Transition**: "Lesson 3.6 summary."

---

# Slide 60: Lesson 3.6 Summary

**Time**: ~1 min **Talking points**:

- Recap: SHAP, LIME, fairness metrics, impossibility theorem, ALE.
- The exercise forces students to move from "the model is accurate" to "the model is accurate, interpretable, and fair." This is the professional standard for deployed ML systems.
- "We now have the full science half. The next two lessons are pure engineering." **Transition**: "Lesson 3.7: automate all of this."

---

# Slide 61: Lesson 3.7 — Workflow Orchestration, Model Registry, and Hyperparameter Search

**Time**: ~1 min **Talking points**:

- Now we move from ML science to ML engineering. Lessons 3.1-3.6 taught what to do. Lessons 3.7-3.8 teach how to automate and productionise it.
- WorkflowBuilder orchestrates the entire pipeline. ModelRegistry tracks versions. HyperparameterSearch finds the best configuration automatically.
- For professionals: this is the layer that separates "ML experiments" from "ML systems that ship." **Transition**: "Start with the orchestrator."

---

# Slide 62: WorkflowBuilder — Node-Based Pipelines

**Time**: ~2 min **Talking points**:

- WorkflowBuilder is from the core Kailash SDK (not kailash-ml). It handles node connections, execution order, error handling, and parallelism.
- Think of it as a factory assembly line: each node does one job and passes the result to the next.
- The pattern: `workflow = WorkflowBuilder(); workflow.add_node(...); workflow.add_connection(...); runtime.execute(workflow.build())`.
- Core concepts: nodes (units of work), connections (data flow), runtime (execution engine).
- Every production ML system at scale uses some variant of this pattern (Airflow, Kubeflow, Metaflow). WorkflowBuilder is Kailash's version.

- If beginners look confused: "A workflow is a recipe written in boxes and arrows. Each box does one thing. The arrows say what goes where."
- If experts look bored: "The runtime supports async execution, retry, caching, and distributed workers out of the box. Same mental model as Airflow DAGs." **Transition**: "Nodes can be built-in or custom."

---

# Slide 63: Custom Nodes

**Time**: ~2 min **Talking points**:

- Custom nodes encapsulate each step of your pipeline.
- `@register_node` decorator registers the node so WorkflowBuilder can find it by name.
- `Node` subclass for full control: define inputs, outputs, execute method.
- `PythonCodeNode` for quick prototyping — wrap any function without subclassing.
- `ConditionalNode` enables branching: if model accuracy above threshold, proceed to registration; otherwise, retrain with different parameters.
- Walk through a simple custom node example: a "preprocess HDB data" node that takes a raw polars DataFrame and returns a cleaned one.
- "In the exercise, students build data loading, preprocessing, training, and evaluation nodes, then wire them into one workflow." **Transition**: "One of those nodes is hyperparameter search."

---

# Slide 64: Bayesian Hyperparameter Search

**Time**: ~2 min **Talking points**:

- HyperparameterSearch wraps Bayesian optimisation with a clean API.
- Grid search and random search are the two classical methods. Bayesian search is smarter: it builds a probabilistic model of the objective function and picks the next point to evaluate based on expected improvement.
- 50 trials with Bayesian optimisation is typically better than 1000 trials with random search.
- SearchSpace uses tuples for ranges: `('float', 0.01, 0.3, 'log')` means a log-scale uniform from 0.01 to 0.3 — right for learning rate.
- Connects to nested CV: the inner loop of nested CV is hyperparameter search.
- If beginners look confused: "Bayesian search is like a chess player who remembers every move they tried and only considers the promising ones next."
- If experts look bored: "Under the hood it is a Gaussian process or TPE. Optuna uses TPE; HyperparameterSearch supports both." **Transition**: "Once you find the best model, you must track it."

---

# Slide 65: Model Registry — Versioning and Lifecycle

**Time**: ~2 min **Talking points**:

- ModelRegistry tracks every model version with its signature, metrics, training data hash, and lifecycle stage.
- ModelSignature defines the contract: what goes in (columns, dtypes), what comes out (label, probability). This prevents the "works on my laptop" problem — a downstream service knows exactly what to send.
- MetricSpec documents which metrics the model reports and what thresholds trigger alerts.
- Lifecycle stages: experiment (dev) -> staging (testing) -> production (live) -> retired (archived).
- "The registry is your audit trail. Every deployed model has a unique version, a signature, and a provenance chain all the way back to the training code."

- Singapore example: MAS model risk management (MRM) guidelines require exactly this kind of versioning for all production models used in regulated decisions. **Transition**: "The lifecycle is more than technical."

---

## Slide 66: Model Lifecycle

**Time**: ~2 min **Talking points**:

- The lifecycle is not just technical; it includes governance gates.
- Promotion from staging to production requires human approval. This is where ML engineering meets organisational process.
- For banks and insurers: this maps to the model risk committee approval workflow. Staging is where validation and independent review happen; production is where the model affects real decisions.
- Retirement is just as important: a model that is no longer used still exists in the registry with its history — so you can reproduce any past decision for audit.
- If beginners look confused: "It is like publishing a book. Draft, edit, publish, eventually retire. Each stage has its own rules."
- If experts look bored: "The governance gate is the non-optional piece. Kailash's PACT framework in M6 extends this with D/T/R accountability grammar." **Transition**: "One last advanced topic: unbiased model selection."

---

## Slide 67: Nested Cross-Validation — Unbiased Model Selection

**Time**: ~2 min **Talking points**:

- Nested CV is the advanced technique that separates rigorous evaluation from naive evaluation.
- Outer loop: k-fold for performance estimation. Inner loop (inside each outer fold): k-fold for hyperparameter search.
- Why nested: if you tune hyperparameters on the same CV you use to estimate performance, you get an optimistically biased estimate. Nested CV prevents this double-use of data.
- Many published papers use non-nested CV and report optimistic numbers. Do not trust a model comparison that was not nested.
- For professionals: when comparing models for production, always use nested CV. Cost: k_outer * k_inner * num_hyperparam_trials model fits. Expensive but honest.
- If beginners look confused: "Think of it as having two separate exam rooms. One for practice, one for the real grade. Never use the same exam for both."
- If experts look bored: "The variance of nested CV is still nontrivial. Repeated nested CV reduces it further. Expensive but gold-standard." **Transition**: "Lesson 3.7 summary."

---

## Slide 68: Lesson 3.7 Summary

**Time**: ~1 min **Talking points**:

- Recap: WorkflowBuilder, custom nodes, Bayesian HP search, ModelRegistry, lifecycle, nested CV.
- The exercise brings everything together into an automated pipeline. Students experience the engineering side of ML: not just training a model but building a system that trains, evaluates, and registers models automatically.
- "One lesson to go. The final piece: drift, DataFlow, and deployment." **Transition**: "Lesson 3.8. Ship it."

---

# Slide 69: Lesson 3.8 — Production Pipeline, DataFlow, Drift, and Deployment

**Time**: ~1 min **Talking points**:

- The final lesson. Everything from 3.1-3.7 was building toward this: a production ML system.
- We cover DataFlow for database persistence, DriftMonitor for monitoring, model cards for documentation, and conformal prediction for uncertainty quantification.
- "By the end of this lesson, you will have a complete mental model of what ships when a bank deploys a credit scoring model." **Transition**: "First, where do predictions go?"

---

# Slide 70: DataFlow — Database Persistence

**Time**: ~2 min **Talking points**:

- DataFlow handles the "where do predictions go?" question. In production, predictions must be stored for audit, monitoring, and business use.
- The decorator-based schema makes it simple: `@db.model` turns a Python class into a database table with automatic CRUD operations.
- Example: `@db.model class Prediction: id, model_version, input_hash, prediction, confidence, timestamp`.
- DataFlow auto-generates: `db.predictions.create(...)`, `.list(...)`, `.get(...)`, `.update(...)`, `.delete(...)`.
- Zero-config: works with SQLite for local dev, PostgreSQL for production, with the same API.
- For beginners: "DataFlow is how the model's predictions get saved to a database so other systems can use them. You write the schema once, and the database operations are free."
- For experts: "DataFlow is Kailash's zero-config data layer. It generates the CRUD nodes automatically from the model class. No ORM boilerplate." **Transition**: "Database operations involve waiting. Python's async syntax is the clean way to handle waits."

---

# Slide 71: Async/Await — Quick Primer

**Time**: ~2 min **Talking points**:

- Many students will not have seen async/await before. Keep it simple.
- `await` means "wait for this but let others proceed." It is like saying "I am ordering coffee, go ahead and take your break while the barista works."
- The database operations use it because real databases take time — and while you wait, the CPU is free to do other work.
- Syntax: `result = await db.predictions.create(...)`. Must be inside an `async def` function.
- For the exercise, students just need to know the syntax pattern. They do not need to understand event loops or coroutines yet.
- If beginners look confused: "Think of await as 'pause here and come back when ready.' The function does not block everything else in the program."
- If experts look bored: "Kailash runtime is async-first. All DataFlow operations are coroutines. In 3.8's exercise, students call them inside an async execute method." **Transition**: "Async is the right abstraction, but connections are finite."

---

# Slide 72: DataFlow — ConnectionManager

**Time**: ~2 min **Talking points**:

- Connection management is a production concern, not a theoretical one.
- In a notebook, leaked connections are annoying. In a production server handling thousands of requests per second, they cause crashes.
- The context manager pattern is the safe default: `async with ConnectionManager() as conn: ...`. Connections are automatically returned to the pool on exit, even if an exception occurs.
- Singapore example: a trading system that opens a new database connection per request will crash during peak hours. Pooling is essential.
- "The exercise uses ConnectionManager in every database operation. Get in the habit." **Transition**: "Once predictions are stored, we monitor for drift."

---

# Slide 73: DriftMonitor — Detecting Model Decay

**Time**: ~2 min **Talking points**:

- Drift is why ML is not "deploy and forget."
- Data drift: feature distributions shift. COVID-19 broke every model trained on pre-pandemic data overnight. Customer behaviour, market prices, and demographic proportions all change.
- Concept drift: the relationship between features and labels shifts. The same features now predict a different outcome.
- For financial models: interest rate changes, regulatory changes, and economic cycles all cause drift. A model trained in 2019 is not valid in 2024 without re-evaluation.
- DriftMonitor detects drift automatically by comparing incoming data distributions against the training distribution.
- If beginners look confused: "Your model learned yesterday. Today's data is different. Drift monitoring tells you when yesterday's knowledge is stale."
- If experts look bored: "We test both feature drift (X distribution) and concept drift (Y given X distribution). The latter is harder to detect because it requires ground truth labels." **Transition**: "Two statistical tools for the job."

---

# Slide 74: PSI and KS Test

**Time**: ~2 min **Talking points**:

- PSI (Population Stability Index) is the industry standard in finance.
- Formula: PSI = sum over bins of (actual_pct - expected_pct) * ln(actual_pct / expected_pct).
- Rule of thumb: PSI below 0.1 = no significant change, 0.1-0.25 = moderate shift, above 0.25 = significant drift, investigate.
- KS (Kolmogorov-Smirnov) test: maximum difference between the empirical CDFs of two samples. Classical statistical approach, non-parametric.
- Both are complementary. PSI bins the distributions and compares proportions. KS looks at the continuous CDFs without binning.
- In practice, monitor both and alert when either crosses the threshold.
- Singapore banking example: credit risk teams at DBS, UOB, and OCBC all use PSI as the primary drift metric for regulatory reporting.
- If beginners look confused: "PSI gives a number for how much a distribution moved. A score above 0.25 means 'call the modelling team'."

- If experts look bored: "PSI is essentially a symmetric KL divergence between two discretised distributions. KS has distributional guarantees that PSI lacks, but PSI is easier to report." **Transition**: "Kailash packages both."

---

## Slide 75: Kailash Bridge — DriftMonitor

**Time**: ~2 min **Talking points**:

- DriftMonitor wraps PSI and KS into a single monitoring framework.
- Configuration: the reference data is the training set. Production data is compared against it on the configured schedule (hourly, daily, weekly).
- DriftSpec defines thresholds per feature. Critical features get tighter thresholds.
- When drift is detected, the system can trigger automatic retraining, alert a human, or downgrade the model status in the registry.
- "The exercise uses DriftMonitor to simulate drift and verify the system detects it before a human would notice." **Transition**: "Monitoring is one piece. Documentation is another."

---

## Slide 76: Model Cards — Documenting Your Model

**Time**: ~2 min **Talking points**:

- Model cards (Mitchell et al., 2019) are the "nutrition label" for ML models.
- Standard sections: intended use, performance across subgroups, limitations, fairness findings, training data characteristics, ethical considerations, contact information.
- They document performance, limitations, and fairness for anyone who uses or governs the model — not just engineers.
- For regulated industries, model cards are increasingly mandatory.
- Singapore example: MAS FEAT guidelines recommend model cards for every ML system in financial services. The EU AI Act effectively mandates them for high-risk systems.
- The exercise requires students to create a model card for their credit scoring model, covering performance, fairness analysis from 3.6, and known limitations.
- If beginners look confused: "A model card is a README for your model, written for regulators and users, not for developers."
- If experts look bored: "Google Model Cards, Hugging Face model cards, and Nvidia's Model Cards++ are the three standards to know. Kailash supports all three export formats." **Transition**: "One more modern technique: conformal prediction."

---

## Slide 77: Conformal Prediction — Uncertainty Quantification

**Time**: ~2 min **Talking points**:

- Conformal prediction is the modern approach to uncertainty quantification.
- Unlike Bayesian methods, it makes no distributional assumptions. The guarantee is finite-sample and distribution-free.
- The output is a prediction interval with a user-specified coverage level. "With 90% probability, the true HDB price is between S$550,000 and S$610,000."
- The algorithm: hold out a calibration set, compute prediction errors, use the quantile of those errors to form intervals around new predictions.

- For professionals: “Instead of saying the HDB price is S$580,000, the model says S$550,000 to S$610,000 with 90% coverage. That is the level of honesty a business can act on.”
- If beginners look confused: “Instead of a single number, the model gives a range. You can trust the range more than the number.”
- If experts look bored: “Split conformal is the simplest variant. Jackknife+ and CV+ trade compute for tighter intervals. Angelopoulos and Bates 2021 is the accessible reference.” **Transition**: “Now let us see the full pipeline.”

---

## Slide 78: The Full Production Pipeline

**Time**: ~2 min **Talking points**:

- This is the capstone view. Every engine from M3 in its place.
- Walk the diagram: ingestion -> FeatureEngineer/FeatureStore -> TrainingPipeline -> HyperparameterSearch -> evaluation -> calibration -> ModelRegistry -> DataFlow persistence -> DriftMonitor -> model card.
- WorkflowBuilder orchestrates all the other engines into an automated production system.
- “This is what the exercise builds. If you can wire this pipeline end-to-end, you can productionise ML models.” **Transition**: “Let us zoom out and look at the discipline that makes this repeatable.”

---

## Slide 79: MLOps — CI/CD for Machine Learning

**Time**: ~2 min **Talking points**:

- MLOps is the emerging discipline for managing ML in production.
- Key insight: ML systems have two axes of change (code AND data) instead of one. Both must be tracked, tested, and deployed systematically.
- CI/CD adaptations: data validation tests, model performance tests, skew tests (train vs serving), drift monitoring as part of the pipeline.
- For professionals: this is the organisational capability that separates ML experiments from ML products. Most teams fail here — they can train a model but cannot deploy it repeatably.
- Practical components: version control (code, data, models), pipelines (training, evaluation, deployment), monitoring (performance, drift), alerting (on degradation), rollback (to a previous model version).
- If beginners look confused: “DevOps for code is hard. MLOps adds data and models on top. Three moving parts instead of one.”
- If experts look bored: “The Google Rules of Machine Learning and the Uber Michelangelo paper are the two foundational references. Kailash’s engines cover most of what those papers describe.” **Transition**: “Lesson 3.8 summary.”

---

## Slide 80: Lesson 3.8 Summary

**Time**: ~1 min **Talking points**:

- Recap: DataFlow for persistence, DriftMonitor for monitoring, model cards for documentation, conformal prediction for uncertainty, MLOps as the discipline that ties it all together.
- This exercise is the capstone: students build the full pipeline from training to monitoring. If they can do this, they can productionise ML models.
- “That is the eight lessons. Let us take stock before the assessment and preview of M4.” **Transition**: “A quick reference: every formula from today.”

# Slide 81: Module 3 — Key Formulas Reference

**Time**: ~1 min **Talking points**:

- Reference slide. Tell students to photograph this or bookmark it.
- Every formula from M3 in one place: bias-variance decomposition, L1/L2/Elastic Net penalties, Gini, information gain, XGBoost split gain, precision/recall/F1, focal loss, Brier score, Shapley value, disparate impact ratio, PSI.
- "You do not need to memorise these for the quiz. You need to know when to apply them." **Transition**: "And the matching engine reference."

---

# Slide 82: Module 3 — Complete Engine Map

**Time**: ~1 min **Talking points**:

- Reference slide. Every Kailash engine introduced in M3 and the lesson where it was covered.
- Use this to find the relevant lesson when you need to revisit an engine later — during the capstone or in real work.
- "Bookmark this. When you are back at work trying to remember 'which engine does probability calibration,' this table has the answer." **Transition**: "Let us see how far you have come."

---

# Slide 83: Module 3 — What You Can Now Do

**Time**: ~2 min **Talking points**:

- Walk through the outcomes. Students started M3 knowing linear regression from M2. Now they have the complete supervised ML toolkit — both halves.
- Science: features, models, evaluation, interpretation, fairness.
- Engineering: workflows, registry, hyperparameter search, persistence, drift, model cards.
- This is the foundation for M4 (unsupervised) and beyond.
- "If you can train a model, evaluate it honestly, interpret it, and deploy it with drift monitoring, you are doing what most production ML teams in Singapore do on any given day."
- If beginners look confused: "That feels like a lot because it is a lot. You do not have to master everything today. The exercises and the notebooks will be there for you to revisit."
- If experts look bored: "The exercises push further than the slides. If you want depth, do exercise 3.7 with nested CV and exercise 3.8 with real conformal intervals." **Transition**: "Now, how we assess all this."

---

# Slide 84: Module 3 — Assessment

**Time**: ~2 min **Talking points**:

- Two components: a quiz for conceptual understanding, and an ML pipeline project for applied competence.
- The project is the capstone. It requires every skill from M3: features, models, evaluation, interpretation, and production deployment.
- Quiz tests concepts. Project tests execution. Together they assess both the science and the engineering.
- Point out the assessment schedule and where to submit.

- "The project is where you put together everything you just learned. Do not wait until the night before." **Transition**: "Here is the exercise schedule."

---

## Slide 85: Exercise Summary

**Time**: ~1 min **Talking points**:

- Quick reference for the exercises.
- Notice the progression: HDB data for features and bias-variance (familiar from M2), then credit scoring for the full pipeline.
- Credit scoring is the running case study from 3.4 through 3.8.
- "Each exercise has three formats: local .py, Jupyter notebook, and Colab. Pick one and stick with it." **Transition**: "One more reference: traps to avoid."

---

## Slide 86: Common Mistakes to Avoid

**Time**: ~2 min **Talking points**:

- These are mistakes professionals actually make in production. Every item on this list has caused a real-world ML failure.
- Typical items: preprocessing leakage, using accuracy on imbalanced data, not calibrating probabilities, k-fold on time series, tuning on the test set, ignoring drift, no model card, no fairness testing.
- The Kailash engines are designed to prevent several of them automatically: TrainingPipeline prevents preprocessing leakage, DriftMonitor prevents silent degradation, ModelRegistry prevents version confusion.
- "Print this list. Stick it next to your monitor. Check it before every ML deployment."
- If beginners look confused: "You will not remember all these today. That is fine. Come back to this list before shipping anything."
- If experts look bored: "Add your own to the list as you gain experience. Every team's top 10 mistakes evolve with their domain." **Transition**: "And here is the decision framework."

---

## Slide 87: Model Selection Decision Tree

**Time**: ~2 min **Talking points**:

- This is the practical decision framework students can use in their projects.
- Not a rigid algorithm, but a starting point.
- Walk through the branches: tabular data -> gradient boosting first; small data -> KNN or linear; need interpretability -> linear or tree; high-dimensional sparse -> SVM or logistic; text -> Naive Bayes baseline then deep models.
- The key message: always try multiple models, always use the right metric, always validate with proper CV.
- "When in doubt, start with XGBoost on tabular data. It is the most frequent winner in our zoo." **Transition**: "One last big picture."

---

## Slide 88: The Complete Supervised Pipeline

**Time**: ~2 min **Talking points**:

- The complete picture. Features flow through models, evaluation, interpretation, orchestration, and production.
- Walk the arrows. Note the dashed feedback loop from DriftMonitor back to retraining — this is the ML lifecycle that every professional ML system follows.
- "This diagram is what M3 has been building toward since slide 1. You now understand every box and every arrow."
- If beginners look confused: "Do not try to memorise this. It is a reference. You will look at it again whenever you plan an ML project."
- If experts look bored: "Extend this mentally to M4 (unsupervised) and M5 (LLMs). The pipeline thinking transfers. Only the evaluation and governance pieces change." **Transition**: "A preview of what comes next."

---

# Slide 89: Coming Up — Module 4

**Time**: ~1 min **Talking points**:

- Brief preview. M4 extends from labelled data (supervised) to unlabelled data (unsupervised).
- The tools from M3 carry forward: WorkflowBuilder, ModelRegistry, DriftMonitor, DataFlow. The pipeline thinking stays the same.
- Evaluation changes: no labels means different metrics (silhouette score, Davies-Bouldin, reconstruction error). We will cover those next module.
- Teaser: clustering, dimensionality reduction, anomaly detection, and the first taste of deep learning. **Transition**: "That wraps up Module 3."

---

# Slide 90: Module 3 Complete

**Time**: ~1 min **Talking points**:

- Thank the class for the session. Three hours is a lot; acknowledge the effort.
- Remind them of the assessment schedule (quiz + ML pipeline project). Point to the exercise repo and the three formats (local, Jupyter, Colab).
- Encourage them to do at least one end-to-end pipeline before the next session.
- If beginners look confused: "You just completed the supervised ML module. Review the formula reference slide and the engine map. If you only remember one thing: always cross-validate, always calibrate, always monitor for drift."
- If experts look bored: "The exercises have depth. Push yourself on 3.7 (nested CV, Bayesian search) and 3.8 (conformal prediction, DriftMonitor with simulated drift)." **Transition**: "See you in Module 4. Go build something."

---

**Timing summary**: Talking points sum to ~170 min across 90 slides, leaving ~10 min of headroom for room transitions, live Q&A pauses, and the short coding demos embedded in Lessons 3.1, 3.4, and 3.8. Total target: ~180 min (3 hours).